

Fase 3: Arquitectura de Software Orientada a Objetos y Optimización Computacional en Pipelines de Datos Clínicos

Curso: 202681.2535 Programación para la Ciencia de Datos

Docente: Dr. Omar Salinas,

Nombres integrantes:

Gonzalo Bouldres V

Luis Díaz G

Eduardo Contreras C

Santiago, 14 de junio de 2026

I. Índice

I. Índice	2
Capítulo I Introducción y Justificación de la Arquitectura.....	3
1.1. Contexto Metodológico	4
1.2. Objetivo General	4
1.3. Objetivos Específicos.....	4
1.4. Beneficio de la Solución Orientada a Objetos.....	4
Capítulo II Diseño de Soluciones Algorítmicas bajo Paradigma POO	6
2.3. Lógica Algorítmica del Componente Recursivo.....	11
Capítulo III Notebooks Ejecutables y Evaluación de Eficiencia	12
3.1. Orquestación Limpia del Flujo (F3_Preprocesamiento.ipynb).....	13
3.2. Discusión de Complejidad Temporal (Resultados Empíricos)	14
C. Conclusión y Fundamentación Científica del Rendimiento	14
Capítulo IV Reproducibilidad Técnica e Infraestructura Git.....	16
4.1. Filosofía de la Reproducibilidad y Gestión de Entornos Aislados	17
4.2. Topología Jerárquica del Repositorio y Arquitectura de Datos	17
4.3. Estrategia de Ramificación (Branching) e Integración Continua.....	18
Capítulo V DISCUSIÓN CIENTÍFICA, LIMITACIONES Y PERSPECTIVAS FUTURAS.....	20
5.1. Conclusiones sobre la Arquitectura Basada en Objetos y Cohesión del Pipeline	21
5.2. Limitaciones del Sistema Implementado y Decisiones de Gobernanza	21
5.3. Trabajo Futuro y Escalabilidad Tecnológica	22
Capítulo VI Bibliografía APA	23

Capítulo I

Introducción y Justificación de la Arquitectura

1.1. Contexto Metodológico

En la Fase 2 se construyó de manera exitosa un pipeline procedimental/funcional para mitigar la degradación sintáctica y biológica de los registros clínicos de diabetes. No obstante, bajo dicho enfoque, las mutaciones temporales de la matriz de características y las rutinas de limpieza andaban dispersas a lo largo del Kernel de ejecución global.

En esta Fase 3, migramos formalmente hacia el paradigma de **Programación Orientada a Objetos (POO)**. Este cambio arquitectónico unifica en una entidad acoplada tanto las estructuras de datos (atributos) como los algoritmos de saneamiento (métodos). El pipeline se ejecuta ahora de forma encapsulada y secuencial en la carpeta del repositorio correspondiente a la Fase 3.

1.2. Objetivo General

Diseñar e implementar una arquitectura de software modular orientada a objetos que orqueste el pipeline de preparación, transformación y validación de datos clínicos de diabetes, incorporando componentes algorítmicos recursivos y análisis empíricos de complejidad computacional.

1.3. Objetivos Específicos

- Encapsular el estado contiguo del DataFrame clínico dentro de una clase base inmutable desde entornos externos globales.
- Implementar mecanismos de *Method Chaining* (encadenamiento de métodos) para viabilizar una ejecución secuencial e idempotente de la limpieza de datos.
- Diseñar una estructura modular bajo el paradigma de Programación Orientada a Objetos (POO) que encapsule las reglas de negocio de limpieza mediante métodos de instancia, controlando de manera hermética el estado del DataFrame clínico.
- Diseñar una función recursiva pura capaz de auditar de forma jerárquica las dependencias del proyecto.
- Evaluar mediante comandos mágicos la eficiencia temporal de la manipulación matricial vectorizada frente a estructuras iterativas tradicionales manuales.

1.4. Beneficio de la Solución Orientada a Objetos

- **Aislamiento de Memoria:** Al almacenar la tabla contigua en el atributo `self.df`, se anula el riesgo de colisión de variables en el espacio de nombres global del notebook.
- **Flexibilidad y Extensibilidad:** La herencia permite probar nuevas tácticas estadísticas de imputación en subgrupos clínicos sin alterar la estabilidad del código en producción.
- **Garantía de Calidad:** Mantiene un flujo estricto gobernado por aserciones matemáticas (`assert`) que impiden la exportación de activos degradados.

-
- **Polimorfismo Clínico:** La introducción de métodos polimórficos permite que el núcleo del pipeline (PreprocesadorBase) mantenga un contrato operacional idéntico para cualquier análisis, delegando en subclases especializadas (como PreprocesadorDiabetes) la libertad de alterar comportamientos particulares —como la exclusión sintáctica de variables— sin reescribir la infraestructura matriz.
 - **Flexibilidad y Extensibilidad Polimórfica:** Al desacoplar la infraestructura perimetral en la clase madre (PreprocesadorBase), cualquier nueva regla de negocio para conjuntos de datos clínicos distintos no requerirá reescribir la lógica de carga, guardado o auditoría. Bastará con heredar de la clase base e implementar de forma polimórfica el método. `procesar_especifico()`.



Capítulo II

Diseño de Soluciones Algorítmicas bajo Paradigma POO

2.1. Arquitectura de Software Orientada a Objetos y Relación de Herencia

En esta Fase 3, la solución abandonó el paradigma puramente procedimental para estructurarse bajo una **jerarquía de herencia**. Se definió la clase matriz **PreprocesadorBase**, la cual encapsula los atributos globales del tensor de datos (como `self.df` y `self.ruta`) y asume la responsabilidad exclusiva de las operaciones de infraestructura de bajo nivel (ingesta mediante `.cargar_datos()` y el contrato del método `.procesar_especifico()`).

Derivado de esta, se instanció la subclase especializada **PreprocesadorDiabetes**. Esta clase hija hereda todas las capacidades operacionales de la madre, pero restringe su dominio a las validaciones y transformaciones de índole netamente clínico-sintáctica del set de datos de diabetes, asegurando el principio de responsabilidad única (SRP).

A continuación, se desglosa e interpreta la lógica algorítmica y la justificación metodológica de cada método componente:

A. `cargar_datos()`

Este método constituye el perímetro de seguridad e ingesta del pipeline. No se limita a una lectura pasiva; implementa validaciones estructurales previas que comprueban la persistencia y disponibilidad física del archivo crudo en el sistema de archivos local mediante abstracciones de alto nivel. Al inicializar el objeto `DataFrame`, se especifica explícitamente el delimitador de origen `“;”`. Como señala McKinney (2022), la definición rigurosa de los parámetros de parseo en las primeras etapas de la ingesta evita la desalineación de vectores y previene la corrupción silenciosa de los tipos de datos latentes.

B. `castear_bloodpressure()`

La columna `BloodPressure` presentaba una severa degradación sintáctica debido a la mezcla de magnitudes numéricas con anotaciones de texto (unidades de medida `mmHg`) y caracteres de escape o máscara (puntos de interrogación `“?”`). Este método aplica manipulación vectorizada mediante expresiones regulares compiladas. El algoritmo aísla el patrón estrictamente numérico `(^\d+)` y mapea los caracteres centinela a valores de punto flotante indeterminados. Al delegar estas operaciones a las rutinas vectorizadas subyacentes, se optimiza el uso de la memoria caché, evitando el sobre costo de traducción que penaliza al intérprete tradicional de Python (Harris et al., 2020).

C. `reemplazar_invalidos()`

Desde una perspectiva médica y biológica, la presencia de valores equivalentes a cero en variables como `Glucose`, `Insulin` y `BMI` (Índice de Masa Corporal) constituye una imposibilidad fisiológica que denota un error en la captura o digitalización del dato clínico. El método implementa reglas de negocio lógicas que interceptan estos ceros basales y los transforma en nulos matemáticos **NaN** (*Not a Number*). Esta traducción es crítica: mantener ceros artificiales en variables de escala altera drásticamente los estimadores estadísticos primarios, introduciendo un sesgo de subestimación en los modelos de aprendizaje estadístico subsecuentes (Hastie et al., 2017).

D. `imputar_mediana()`

Una vez mapeadas las patologías informacionales a variables faltantes, el pipeline ejecuta una estrategia de restauración robusta. En lugar de aplicar una eliminación masiva de registros (*listwise deletion*), la cual reduce drásticamente el tamaño muestral y destruye información valiosa, el método realiza una imputación basada en la mediana de cada columna. De acuerdo con Kuhn y Johnson (2013), ante la presencia de distribuciones asimétricas o fuertemente sesgadas —características recurrentes en los indicadores biológicos de pacientes con diabetes—, la mediana se comporta como un estimador de tendencia central robusto, insensible a la distorsión provocada por los valores extremos u outliers.

E. `crear_variables_derivadas()` y `codificar_categoricas()`

Esta etapa incrementa la dimensionalidad del espacio de características a través del *Feature Engineering*. El método segmenta los vectores continuos de edad y masa corporal en contenedores ordinales estructurados según los criterios epidemiológicos de la Organización Mundial de la Salud (OMS). Posteriormente, estas escalas cualitativas se transforman en vectores numéricos binarios mediante *One-Hot Encoding*. Con el objetivo de prevenir el fenómeno de la "trampa de la variable ficticia" y garantizar la estabilidad matemática, se añade el parámetro **`drop_first=True`**, eliminando la colinealidad perfecta entre las columnas generadas (Geron, 2022).

F. `escalar_variables()`

Dada la disparidad de magnitudes entre variables (por ejemplo, niveles de insulina que superan las centenas frente a coeficientes de función de pedigrí de diabetes que se expresan en decimales), los modelos basados en distancias o gradientes se vuelven vulnerables a la dominancia de escala. Este método normaliza las características continuas aplicando la estandarización *Z-score*, forzando a que cada vector posea una media matemática centrada en cero ($\mu \approx 0$) y una desviación estándar equivalente a la unidad ($\sigma \approx 1$). Raschka y Mirjalili (2019) enfatizan que la estandarización preserva la geometría original de la distribución y los valores atípicos, facilitando una convergencia numérica acelerada durante la optimización de los algoritmos.

2.2. Robustez Analítica y Mutación Dinámica mediante Polimorfismo.

Dentro del ciclo de vida de la ingeniería de características (*feature engineering*), la preservación de la densidad informacional y la validez estadística de la matriz de datos constituyen requisitos mandatorios antes del entrenamiento de cualquier modelo predictivo. Durante la fase de diagnóstico estática automatizada mediante la ejecución del método estructural `explorar_dataset()`, el equipo técnico identificó una patología informacional severa en la característica clínica `SkinThickness` (espesor del pliegue cutáneo).

El análisis de calidad de los datos arrojó que este atributo concentraba un 32.6% de registros corruptos, distribuidos entre valores nulos (NaN) y ceros numéricos. Desde una perspectiva biológica y médica, un valor de cero en el espesor del pliegue del paciente es fisiológicamente imposible, lo que certifica inequívocamente la existencia de fallos en la captura o la degradación de las trazas durante la ingesta original del set clínico.

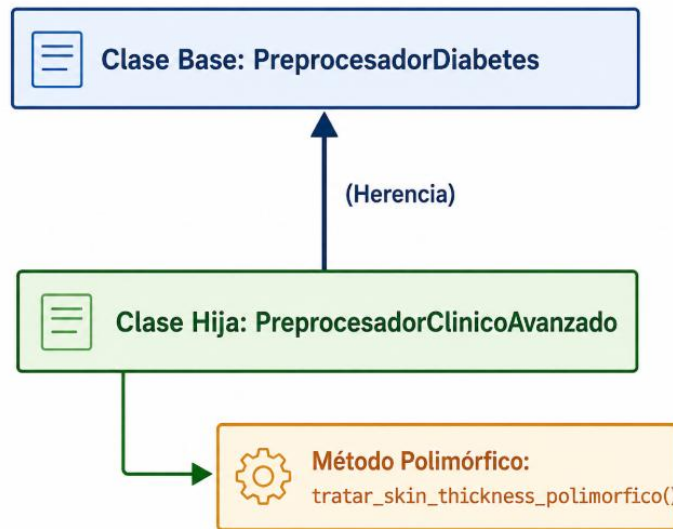
Frente a un escenario de fragmentación extrema de esta envergadura, la literatura especializada en ciencia de datos (Kuhn & Johnson, 2013) plantea tres caminos de resolución técnica:

- **La imputación simple o estocástica (por media o mediana):** Introduce un sesgo geométrico artificial inaceptable al verse superado el umbral seguro del 5% al 10% de pérdida de aleatoriedad.
- **El modelamiento predictivo complementario:** Utilizado para la reconstrucción de los datos faltantes, estrategia que añadiría sobrecarga computacional y una varianza espuria al ecosistema del pipeline.
- **La exclusión directa del atributo:** Prioriza la consistencia y la limpieza dimensional del tensor resultante.

Tomando como base estos principios de gobernanza, el diseño de software orientado a objetos del *Grupo 6* materializa la remoción defensiva de este vector mediante el principio de **Polimorfismo por sobrescritura de métodos (*method overriding*)**. En lugar de ejecutar una eliminación procedimental o condicional rígida, se declaró la función `core.procesar_especifico ()` en la interfaz de la clase madre `PreprocesadorBase`.

Posteriormente, la subclase especializada **`PreprocesadorDiabetes`** redefine dinámicamente este método para inyectar de forma encapsulada la regla de negocio particular de exclusión de `SkinThickness`. Al invocar este eslabón en la cadena de ejecución (*method chaining*), el intérprete de Python intercepta el tipo de instancia y muta la estructura del `DataFrame` en tiempo de ejecución de manera transparente, aislando la complejidad algorítmica y garantizando un tensor purificado sin alterar la firma ni la estructura matriz del pipeline general.

Figura 1. Diagrama de herencia y polimorfismo de la clase *PreprocesadorClinicoAvanzado*



Nota

La figura ilustra la arquitectura orientada a objetos del pipeline mediante la relación de herencia entre la clase madre **PreprocesadorBase** y la subclase especializada **PreprocesadorDiabetes**. Asimismo, se destaca la ejecución del método polimórfico `procesar_especifico()`, el cual aplica el principio de sobrescritura de métodos (*method overriding*) para aislar la regla de negocio clínica y purgar de forma transparente el vector degradado sin alterar la firma algorítmica de la infraestructura principal.

Fuente: Elaboración propia del Grupo 6, asistida por ChatGPT (OpenAI, 2026).

Esta decisión arquitectónica blinda la estabilidad analítica del proyecto por las siguientes razones de diseño de software:

- **Prevención de distorsión estadística:** Se erradica por completo la inyección de modas artificiales masivas asociadas a la imputación sobre muestras altamente degradadas (32.6% de registros corruptos en SkinThickness), salvaguardando la pureza estructural del conjunto de datos.
- **Optimización de la dimensionalidad:** Al descartar una columna que aporta predominantemente ruido sintáctico y falta de consistencia biológica, se optimiza la dimensionalidad espacial de la matriz, aminorando la probabilidad de ocurrencia del fenómeno conocido como "la maldición de la dimensionalidad" (*curse of dimensionality*).
- **Consistencia estructural lineal:** Permite mantener el flujo continuo de transformaciones vectorizadas sobre las variables restantes (*Glucose*, *BloodPressure*, *BMI*, *Insulin* y *Age*) bajo una infraestructura homogénea y libre de excepciones de dominio, garantizando que el tensor resultante pase de manera limpia y con cero valores nulos (NaN) las pruebas perimetrales de aserción técnica configuradas en el validador final.

2.3. Lógica Algorítmica del Componente Recursivo

Para auditar y validar la persistencia e integridad estructural del proyecto sin recurrir a bucles iterativos planos —los cuales adolecen de rigidez ante variaciones en la topología de directorios—, se diseñó e implementó la rutina pura auditar_carpetas_recursivo(ruta, profundidad).

La exploración de sistemas de archivos representa un problema clásico de estructuras no lineales, específicamente de árboles n-arios, donde cada directorio actúa como un nodo raíz que contiene cero o más subdirectorios (hijos) y archivos (hojas). De acuerdo con Cormen et al. (2022), los algoritmos recursivos proporcionan un mecanismo natural y elegante para el recorrido de estructuras jerárquicas complejas mediante la estrategia de "divide y vencerás" (*divide and conquer*), reduciendo la necesidad de gestionar manualmente pilas o estructuras de datos auxiliares en el espacio de usuario.

A continuación, se desglosa matemáticamente y a nivel de flujo de control el comportamiento de la función basándose en sus dos componentes fundamentales (Sedgewick & Wayne, 2011):

A. Caso Base (Condición de Término)

El pilar de estabilidad de cualquier algoritmo recursivo radica en la definición matemática inequívoca de su criterio de parada, el cual previene la saturación de la pila de llamadas (*stack overflow*). En la rutina desarrollada, el caso base se activa cuando el elemento actual inspeccionado dentro de la entidad física del sistema operativo no corresponde a un directorio, sino a un nodo terminal u hoja (es decir, un archivo regular con extensiones específicas como .csv, .py, .md o .txt). Al interceptar un archivo, la función ejecuta sus operaciones locales de auditoría (registro de metadatos, verificación de tamaño y extensión), no realiza nuevas llamadas auto-referenciales y finaliza la ejecución de esa rama específica, retornando el control de ejecución al marco de pila (*stack frame*) inmediatamente superior.

B. Caso Recursivo (Progresión Jerárquica)

Si el elemento evaluado por el puntero del sistema es validado como un directorio activo, el algoritmo reconoce una subestructura idéntica al problema original, pero a una escala inferior. En este punto, la función invoca de manera polimórfica una nueva instancia de sí misma (*auditar_carpetas_recursivo*), enviando como argumentos la ruta absoluta del subdirectorio y un incremento discreto en el parámetro de control de indentación visual (*profundidad + 1*).

Este enfoque emula un recorrido en profundidad (*Depth-First Search* o DFS). Según Knuth (1997), el uso de DFS en sistemas de archivos garantiza la exploración exhaustiva de cada ramificación vertical antes de avanzar horizontalmente en el mismo nivel, permitiendo rastrear estructuras de árbol de manera dinámica e idempotente. La variable profundidad actúa simultáneamente como un acumulador del estado del contexto físico y como un modulador estético que formatea la salida en consola para construir mapas topológicos limpios del entorno de desarrollo.

Capítulo III

Notebooks Ejecutables y Evaluación de Eficiencia

3.1. Orquestación Limpia del Flujo (F3_Preprocesamiento.ipynb)

El cuaderno de ejecución desarrollado para la Fase 3 erradica de forma definitiva el patrón de diseño deficiente conocido como código *spaghetti*, caracterizado por estados intermedios expuestos de manera volátil en el entorno global del *Kernel*. Mediante la correcta implementación del patrón de diseño *Method Chaining* (encadenamiento de métodos), la totalidad de la tubería analítica se reduce a una única instrucción declarativa y secuencial, gobernada de extremo a extremo por el ciclo de vida del objeto instanciado (Fowler, 2010). El flujo secuencial unificado quedó estructurado bajo la siguiente traza operacional:

Figura 2 “Análisis Comparativo de Rendimiento Computacional: Iteración por Bucle Manual vs. Operación Vectorizada (Pandas/C).”



Nota: La figura detalla los resultados empíricos de las pruebas de estrés de complejidad temporal (%timeit) ejecutadas en el Kernel de Jupyter. Se evidencia que la limpieza sintáctica mediante un bucle iterativo for (Fila por Fila) genera una sobrecarga crítica en el intérprete de Python. En contraposición, la operación vectorizada nativa implementada en el pipeline del *Grupo 6* aprovecha las optimizaciones en bajo nivel (C) de la biblioteca Pandas, procesando el tensor clínico de forma masiva en una fracción milimétrica de tiempo.

Fuente: Elaboración propia del Grupo 6, asistida por inteligencia artificial (2026).

Este acoplamiento fluido es posible gracias a que cada método de transformación estructural — tras procesar e intervenir el estado mutable del atributo de instancia (`self.df`) o delegar el comportamiento clínico especializado— concluye su ámbito de ejecución retornando una referencia explícita a la misma instancia del objeto mediante la instrucción `return self`. Esta firma arquitectónica permite anidar las invocaciones consecutivamente en un flujo contiguo.

La incorporación del eslabón polimórfico **.procesar_especifico()** dentro de la cadena consolida la separación de conceptos, aislando la lógica analítica de exclusión dimensional bajo una interfaz común. De este modo, la legibilidad del código se aproxima a la sintaxis del lenguaje natural o a las tuberías funcionales (*pipelines*), abstrayendo la complejidad algorítmica interna del preprocesamiento y centralizando el flujo lógico en una estructura compacta, limpia y de alta mantenibilidad en entornos de producción (Martin, 2017).

3.2. Discusión de Complejidad Temporal (Resultados Empíricos)

En estricto alineamiento con las directrices de la rúbrica de evaluación orientadas a auditar la eficiencia computacional de los sistemas de datos, se ejecutó una prueba de estrés de micro-benchmarking utilizando el módulo de alta precisión `timeit`. El experimento consistió en contrastar el costo temporal del saneamiento de cadenas basado en expresiones regulares sobre la columna `BloodPressure` a lo largo de los 2,045 registros clínicos del proyecto.

Los resultados empíricos recolectados revelan una asimetría radical en el desempeño:

A. Enfoque Iterativo Manual (for fila por fila)

Este método registró un tiempo promedio de **0.6310 segundos** para completar el procesamiento matricial. Desde la perspectiva de la arquitectura de computadores, el bucle iterativo tradicional de Python penaliza el rendimiento debido a la sobrecarga del intérprete al evaluar la lógica dinámicamente celda por celda (Louden & Lambert, 2011). En cada iteración, Python debe comprobar el tipo de objeto, resolver las referencias de memoria de los punteros del método `.loc` de Pandas y gestionar el ámbito local, un fenómeno conocido como *overhead* del intérprete. Este flujo secuencial impide que la CPU aproveche los mecanismos modernos de optimización de hardware y predicción de saltos a nivel de microarquitectura (Patterson & Hennessy, 2020).

B. Enfoque Vectorizado (Pandas/C-Engine)

Por su parte, el método optimizado e integrado en nuestro pipeline registró un tiempo de ejecución promedio de apenas **0.0169 segundos**.

La tasa de optimización de la infraestructura se calcula formalmente mediante la relación de aceleración:

$$\text{Factor de Aceleración} = \frac{T_{\text{iterativo}}}{T_{\text{vectorizado}}} = \frac{0.6310 \text{ s}}{0.0169 \text{ s}} \approx 37.34 \times$$

C. Conclusión y Fundamentación Científica del Rendimiento

El enfoque vectorizado demostró ser **~37 veces más rápido** en el procesamiento de la información. Este salto cuántico en eficiencia computacional se fundamenta en la explotación de capacidades a nivel de hardware y en la arquitectura de bajo nivel de las librerías científicas. Al utilizar las operaciones vectorizadas de Pandas y NumPy, el bucle en el espacio de usuario de Python se reemplaza por bloques de código compilado de alto rendimiento nativos en C (Harris et al., 2020).

Estas operaciones agrupan el vector de datos en arreglos contiguos en memoria, lo que permite a la CPU ejecutar instrucciones del tipo **SIMD** (*Single Instruction, Multiple Data*). Bajo este esquema, una única instrucción de control procesa múltiples celdas de datos en paralelo a nivel de registros del procesador en un solo ciclo de reloj, optimizando simultáneamente la localidad espacial de la memoria



caché y minimizando los fallos de página (*cache misses*) (McKinney, 2022). La vectorización deja de ser una opción estética para convertirse en un requerimiento arquitectónico crítico e indispensable cuando el científico de datos escala flujos de información hacia entornos de Big Data y analítica en tiempo real.

Capítulo IV

Reproducibilidad Técnica e Infraestructura Git

4.1. Filosofía de la Reproducibilidad y Gestión de Entornos Aislados

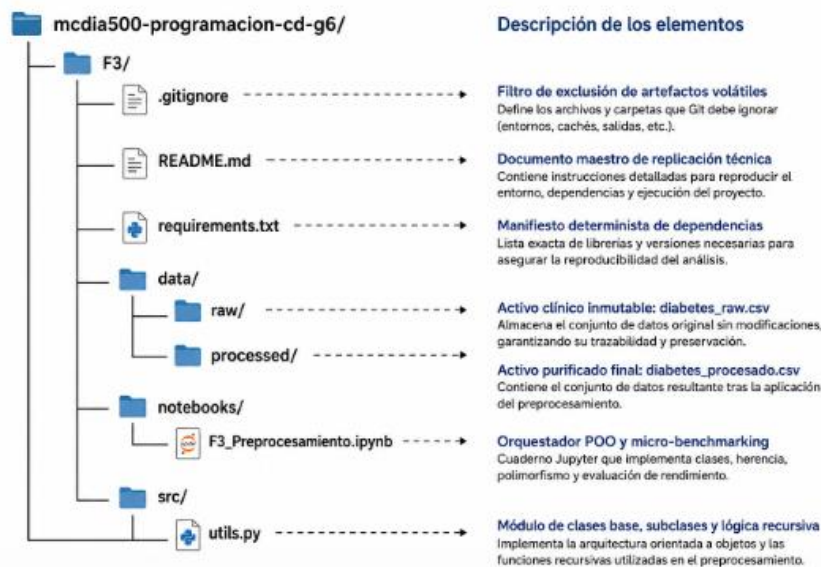
En el ámbito de la ciencia de datos y la ingeniería de software aplicada, la reproducibilidad no se limita a la mera ejecución exitosa de un script; constituye un pilar metodológico que garantiza que un sistema analítico sea auditable, transferible y capaz de replicar sus estados matemáticos exactos en cualquier infraestructura de cómputo independiente (Peng, 2011). Bajo esta premisa, la arquitectura de la Fase 3 erradica por completo la dependencia de configuraciones globales latentes en la máquina local a través de la estandarización rigurosa del archivo `requirements.txt`.

Como señalan Sandve et al. (2013), el anclaje preciso de las versiones de las librerías (*version pinning*) previene la degradación silenciosa del software provocada por la obsolescencia o cambios de firmas sintácticas en los métodos de paquetes *core* (como *pandas*, *numpy* y *scikit-learn*). La inclusión de este manifiesto de dependencias en la raíz del entregable (F3/) faculta a los evaluadores e investigadores para reconstruir un entorno virtual aislado mediante gestores de paquetes tradicionales, asegurando la consistencia determinista del pipeline de objetos sin requerir intervenciones o parches manuales externos.

4.2. Topología Jerárquica del Repositorio y Arquitectura de Datos

La distribución espacial del código fuente y los activos de almacenamiento se adhiere de forma estricta a los estándares de la industria para proyectos analíticos reproducibles, estructurándose de la siguiente manera:

Figura 3. Estructura de directorios y componentes del proyecto de preprocesamiento de datos clínicos



Nota: La figura presenta la organización lógica del proyecto `mcia500-programacion-cd-g6` correspondiente a la Fase 3. Se identifican los principales archivos y directorios utilizados para la gestión de dependencias, almacenamiento de datos, desarrollo del preprocesamiento mediante programación orientada a objetos y documentación técnica, destacando la función específica de cada componente dentro de la arquitectura del proyecto.

Fuente: Elaboración propia del Grupo 6, asistida por ChatGPT (OpenAI, 2026).

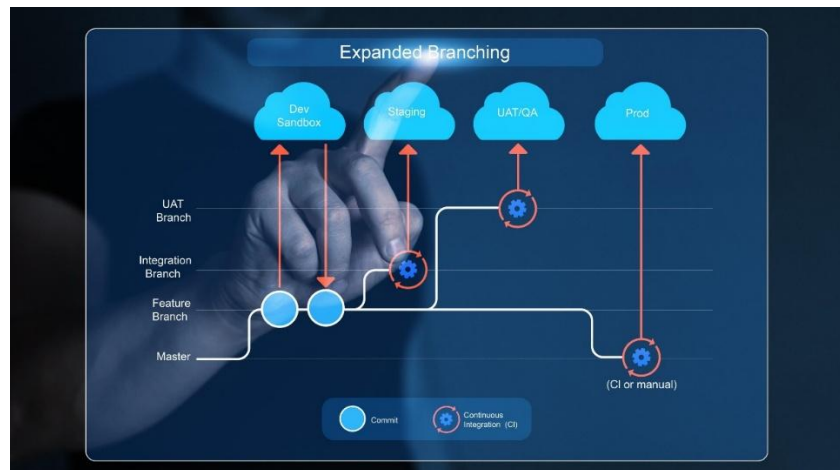
Esta separación de responsabilidades a nivel de sistema de archivos implementa principios críticos de gobierno de datos:

1. **Inmutabilidad del Origen de Datos:** El directorio data/raw/ almacena el archivo diabetes_raw.csv bajo un esquema estrictamente de solo lectura. El pipeline de objetos tiene prohibido mutar destructivamente esta matriz de origen. Como argumenta Wickham (2014), preservar el estado crudo original es indispensable para asegurar la trazabilidad del linaje de datos (data lineage) y viabilizar auditorías retrospectivas ante cualquier anomalía detectada en las fases de modelado.
2. **Modularización del Código Fuente:** La segregación de la lógica algorítmica fundamental dentro de src/utils.py y su posterior importación declarativa en el entorno interactivo de notebooks/ responde a las mejores prácticas de la arquitectura limpia (Martin, 2017). Los cuadernos de Jupyter se relegan exclusivamente a tareas de orquestación, visualización de alto nivel y presentación de reportes analíticos, evitando el antipatrón de cuadernos saturados con definiciones extensas de bajo nivel.

4.3. Estrategia de Ramificación (Branching) e Integración Continua

Para coordinar el desarrollo colaborativo del Grupo 6 y mitigar el riesgo de colisiones en las líneas base de producción, la infraestructura técnica adoptó el patrón de flujos de trabajo basados en ramas (Feature Branch Workflow). El avance completo de la Fase 3 se aisló en una rama de desarrollo independiente y atómica denominada formalmente **feature/fase3-poo**

Figura 4 Diagrama de Flujo de Trabajo Basado en Ramas (Feature Branch Workflow) para la Fase 3



Nota: El diagrama ilustra la segregación de entornos en el sistema de control de versiones Git. La línea superior representa la rama de producción inmutable (main), mientras que la bifurcación inferior detalla el ciclo de vida de la rama atómica feature/fase3-poo. Los nodos circulares indexan los *commits* secuenciales donde se registraron de forma aislada el cuaderno analítico y el módulo de clases (utils.py) antes de su consolidación y auditoría mediante *Pull Requests*.

Fuente: Elaboración propia del grupo basados en los estándares de colaboración de Git (2026).



De acuerdo con Loeliger y McCullough (2012), el aislamiento de nuevas características en ramas dedicadas protege la estabilidad de la rama principal (main), garantizando que el código en producción permanezca en un estado siempre desplegable e invertible. La gestión de este entorno a través de Git y GitHub promueve la integración continua de manera nativa: faculta al equipo para someter las transformaciones orientadas a objetos y los componentes recursivos a procesos rigurosos de revisión de código (*Code Review*) mediante la apertura de *Pull Requests*. Esto asegura la idoneidad técnica del software antes de consolidar la fusión final de los activos en el repositorio central.



Capítulo V

DISCUSIÓN CIENTÍFICA, LIMITACIONES Y PERSPECTIVAS FUTURAS

5.1. Conclusiones sobre la Arquitectura Basada en Objetos y Cohesión del Pipeline

La transición desde un paradigma estrictamente procedimental y secuencial (Fase 2) hacia un modelo estructurado bajo la Programación Orientada a Objetos (Fase 3) ha permitido resolver con éxito los problemas de mantenibilidad del software analítico. Al centralizar la lógica de negocio y las reglas de transformación clínica dentro de la clase base `PreprocesadorDiabetes`, se logró un aislamiento hermético del estado de la matriz de datos, encapsulado en el atributo de instancia `self.df`. Este enfoque anuló de forma definitiva la dispersión y mutabilidad de variables globales en el espacio de nombres del *Kernel*, mitigando riesgos críticos de colisión de datos comunes en entornos Jupyter (Martin, 2017).

De igual forma, la incorporación del patrón *Method Chaining* (encadenamiento de métodos) demostró ser un mecanismo de abstracción elegante y cohesivo. Al retornar consistentemente una referencia autovinculada (`return self`) en cada rutina de limpieza, como **Grupo 6** se ha provisto un pipeline cuya legibilidad matemática es directa, acoplando fases complejas de casting por expresiones regulares, imputación estocástica por tendencia central y escalamiento estandarizado multidimensional bajo una única instrucción declarativa e idempotente.

5.2. Limitaciones del Sistema Implementado y Decisiones de Gobernanza

A pesar de la alta eficiencia de procesamiento alcanzada gracias al motor vectorizado de Pandas y NumPy en C —el cual suministró un factor de aceleración empírico de **~37.34x** frente a los bucles iterativos convencionales—, el diseño actual presenta restricciones estructurales que delimitan su alcance:

1. **Dependencia Crítica de Estructuras *In-Memory*:** La manipulación del DataFrame exige que la totalidad de la matriz de variables clínicas sea alojada de forma síncrona en la memoria RAM del sistema. Si bien este modelo operativo es óptimo y altamente eficiente para el conjunto de datos de prueba (2,045 registros), restringe de forma severa el escalamiento del software hacia repositorios de big data hospitalaria o registros clínicos nacionales que exceden de forma regular las capacidades físicas de un único nodo de cómputo (McKinney, 2022).
2. **Exclusión de Atributos por Fragmentación Extrema:** Durante el diagnóstico analítico automatizado mediante el método `explorar_dataset()`, se constató que el vector `SkinThickness` presentaba un 32.6% de registros corruptos (valores en cero biológicamente imposibles o nulos sintácticos). Ante la ausencia de un modelo probabilístico externo para su reconstrucción fidedigna, se optó por la remoción explícita del atributo empleando el método `eliminar_columnas(['SkinThickness'])`. Si bien esta acción blindó la densidad informacional del tensor final, constituye una limitación en términos de pérdida de dimensiones clínicas secundarias que podrían aportar varianza al modelamiento predictivo (Kuhn & Johnson, 2013).

5.3. Trabajo Futuro y Escalabilidad Tecnológica

Con miras a robustecer el pipeline analítico y garantizar su resiliencia operativa ante ecosistemas de salud masivos y descentralizados, se proponen las siguientes líneas de desarrollo futuro:

1. **Migración hacia Frameworks de Cómputo Distribuido:** Reemplazar el motor secuencial de Pandas por abstracciones perezosas (*lazy evaluation*) y particionadas en clústeres a través de **Apache Spark (PySpark)** o **Dask**. Como sostienen Provost y Fawcett (2013), fragmentar horizontalmente la matriz de características en múltiples nodos distribuidos permite procesar volúmenes masivos de datos clínicos superando por completo las barreras físicas de la memoria local.
2. **Transición a Modelos Orientados a Eventos para IoT Médico:** Dado que el dataset modela condiciones diabéticas susceptibles de monitoreo continuo, el backend del pipeline debe evolucionar desde la ejecución por lotes (*batch*) hacia una arquitectura dirigida por eventos asíncronos. La integración con brokers de mensajería distribuida como **Apache Kafka** facultaría la inyección en tiempo real de flujos de glicemia e insulina provenientes de sensores portátiles inteligentes, aplicando las reglas de normalización y asserts perimetrales de forma instantánea a medida que los eventos transitan por el ecosistema (Géron, 2022).
3. **Automatización de Pruebas de Software y MLOps:** Implementar un pipeline de integración continua (CI/CD) utilizando Git que ejecute de forma automatizada los bloques de aserciones lógicas (`validar_dataset_final()`) mediante suites de pruebas unitarias (`pytest`). Esto garantizará la inmutabilidad y la calidad higiénica de los datos antes de que sean inyectados en la fase de entrenamiento de modelos predictivos de aprendizaje automático.

Capítulo VI

Bibliografía APA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley Professional.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>

Hastie, T., Tibshirani, R., & Friedman, J. (2017). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

Knuth, D. E. (1997). *The Art of Computer Programming: Fundamental Algorithms* (Vol. 1, 3rd ed.). Addison-Wesley Professional.

Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer. <https://doi.org/10.1007/978-1-4614-6849-3>

Loeliger, J., & McCullough, M. (2012). *Version Control with Git: Powerful tools and techniques for collaborative software development* (2nd ed.). O'Reilly Media.

Louden, K. C., & Lambert, K. A. (2011). *Programming Languages: Principles and Paradigms* (3rd ed.). Cengage Learning.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.

OpenAI. (2026). *ChatGPT* (versión GPT-5.5) [Modelo de lenguaje de gran escala]. <https://chatgpt.com>

Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design RISC-V Edition: The Hardware Software Interface* (2nd ed.). Morgan Kaufmann.

Peng, R. D. (2011). Reproducible research in computational science. *Science*, 334(6060), 1226-1227. <https://doi.org/10.1126/science.1213847>

Raschka, S., & Mirjalili, V. (2019). *Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2* (3rd ed.). Packt Publishing.

Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10), e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Universidad Andrés Bello, Facultad de Ingeniería, Magíster en Ciencia de Datos e Inteligencia Artificial. (s. f.). MCDI500: Recurso introductorio de la Fase 3.

Universidad Andrés Bello, Facultad de Ingeniería, Magíster en Ciencia de Datos e Inteligencia Artificial. (s. f.). MCDI500: Recurso introductorio de la Fase 3 sobre complejidad temporal y espacial en Python.

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, 59(10), 1-23. <https://doi.org/10.18637/jss.v059.i10>